E-Commerce Platform with Distributed Transactions

Implementation, Observability, and Event-Driven Autoscaling Report

April 9, 2026

Abstract

The current report highlights the design and implementation process of a robust E-Commerce microservices system, based on 2-Phase Commit architecture. Although the key design considerations relate to handling distributed transactions using Java Spring Boot framework, the present document provides a significant focus on the observability stack, which includes Prometheus, Grafana, and Jaeger, along with the event-driven auto-scaling process using Kubernetes Event-driven Autoscaling (KEDA).

Contents

1 Introduction and Objectives **2**
1.1 Project Overview . . . 2
1.2 Objective . . . 2

2 Architecture Overview **2**
2.1 System Architecture . . . 2
2.2 Technology Stack . . . 2

3 Core Development: Microservices & Distributed Transactions **3**
3.1 The 2-Phase Commit (2PC) Implementation . . . 3
3.2 Database Provisioning . . . 3

4 Kubernetes Deployment Strategy **3**
4.1 Stateless Workloads . . . 3
4.2 Stateful Workloads . . . 3

5 Observability: Monitoring & Distributed Tracing **3**
5.1 Metrics Collection with Prometheus . . . 4
5.2 Distributed Tracing with Jaeger . . . 4
5.3 Data Visualization with Grafana . . . 4

6 Dynamic Autoscaling with KEDA & Prometheus **6**
6.1 HTTP Traffic-Based Scaling (Coordinator) . . . 6
6.2 Kafka Lag-Based Scaling (Worker Services) . . . 6

7 Conclusion and Future Enhancements **6**
7.1 Conclusion . . . 6
7.2 Future Enhancements . . . 7

1 Introduction and Objectives

1.1 Project Overview

E-Commerce is implemented as a distributed application consisting of various microservices. Monolithic applications provide intrinsic support for ACID transactions, but in a distributed scenario, to ensure that an order, an update in inventory, and a payment transaction occur in a consistent manner, a distributed transaction is needed.

1.2 Objective

The main aim of this assignment is to apply the 2-Phase Commit (2PC) design pattern among different microservices in order to guarantee consistency in data. In addition to this, in order to guarantee high availability and high responsiveness to workload, proper monitoring strategies and smart autoscaling mechanisms based on performance metrics have been set up.

2 Architecture Overview

2.1 System Architecture

The application ecosystem spans several logical planes: the application plane (microservices), the messaging plane (Kafka), the data plane (PostgreSQL), and the control/observability plane (Kubernetes, Prometheus, Jaeger).

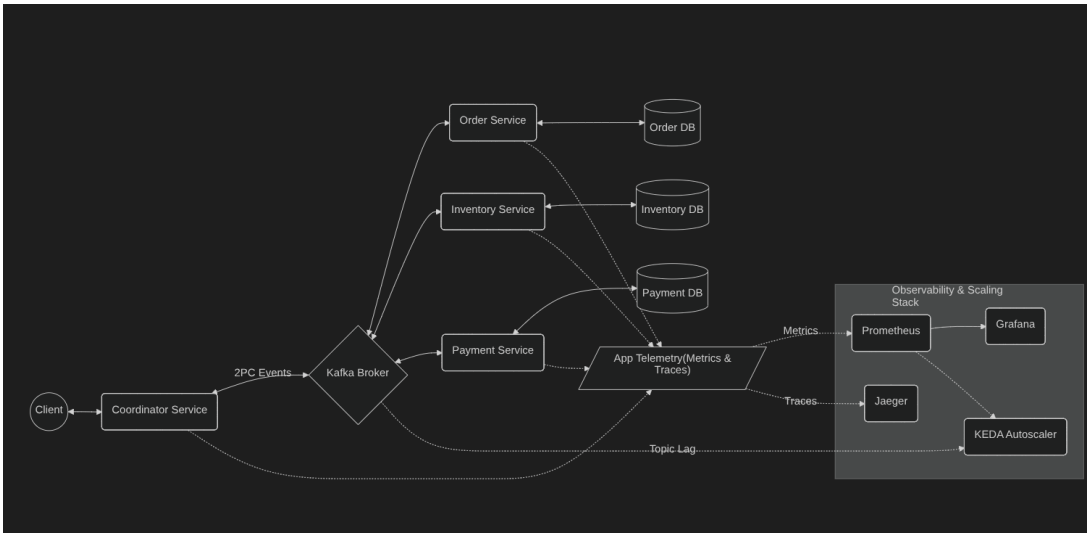

Figure 1: *High-Level Architecture Diagram*

2.2 Technology Stack

- **Backend Core:** Java 21, Spring Boot 3.5.x
- **Messaging and Eventing:** Apache Kafka and Zookeeper
- **Databases:** PostgreSQL

- **Container Orchestration:** Kubernetes (Minikube)
- **Observability:** Prometheus, Grafana, Jaeger (OTLP)
- **Autoscaling:** KEDA (Kubernetes Event-driven Autoscaling)

3 Core Development: Microservices & Distributed Transactions

3.1 The 2-Phase Commit (2PC) Implementation

The business logic encapsulates the traditional saga/2PC workflows. The transaction lifecycle is managed centrally to assure atomicity across boundaries.

- **Coordinator Service:** Acts as the transaction manager. It orchestrates the *Prepare* phase, awaiting votes from participating services, and subsequently executes the *Commit* or *Abort* phase based on the consensus.
- **Participating Services:** The `order-service`, `inventory-service`, and `payment-service` execute local transactions and lock resources during the prepare phase, communicating their readiness back to the Coordinator.
- **Event-Driven Communication:** Using the shared `events` module, services communicate asynchronously via Kafka topics, heavily decoupling the microservices and enhancing fault tolerance.

3.2 Database Provisioning

Stateful persistence is managed securely via Kubernetes ConfigMaps and Secrets.
The `postgresql-initdb-config.yml` automatically provisions schemas and databases for each isolated microservice domain upon initialization, guaranteeing separation of concerns at the data level.

4 Kubernetes Deployment Strategy

4.1 Stateless Workloads

The Spring Boot web applications (`coordinator`, `order`, `inventory`, `payment`) are deployed as standard Kubernetes Deployments exposed via Services. This structure enables them to scale horizontally in a stateless manner.

4.2 Stateful Workloads

Data tier components require stable network identities and persistent storage.

- **PostgreSQL:** Deployed using StatefulSets to ensure data durability and ordered automated rolling updates.
- **Kafka and Zookeeper:** Similarly deployed as StatefulSets, guaranteeing ordered startup and stable clustering necessary for reliable message brokering.

5 Observability: Monitoring & Distributed Tracing

This section highlights the critical observability features that provide deep visibility into the cluster's behavior.

5.1 Metrics Collection with Prometheus

Prometheus acts as the central time-series database. Through the `prometheus-config.yml`, it is configured to dynamically scrape the `/actuator/prometheus` endpoints exposed by each Spring Boot 3 service every 5 seconds. Additionally, a dedicated `kafka-exporter` captures granular metrics from the Kafka brokers regarding topic health and consumer groups.

5.2 Distributed Tracing with Jaeger

To trace the lifecycle of a single user request traversing the Coordinator, Order, Inventory, and Payment systems, OpenTelemetry (OTLP) has been integrated. Traces are exported to the Jaeger backend, enabling the identification of latency bottlenecks and visualizing failures during a distributed transaction rollback.

Figure 2: *Jaeger Trace showing a successful 2PC transaction flow*

5.3 Data Visualization with Grafana

Using ConfigMaps, detailed Grafana dashboards are pre-loaded into the cluster. This provides immediate, out-of-the-box monitoring for JVM insights, Spring Boot request rates, and Kafka pipeline health.

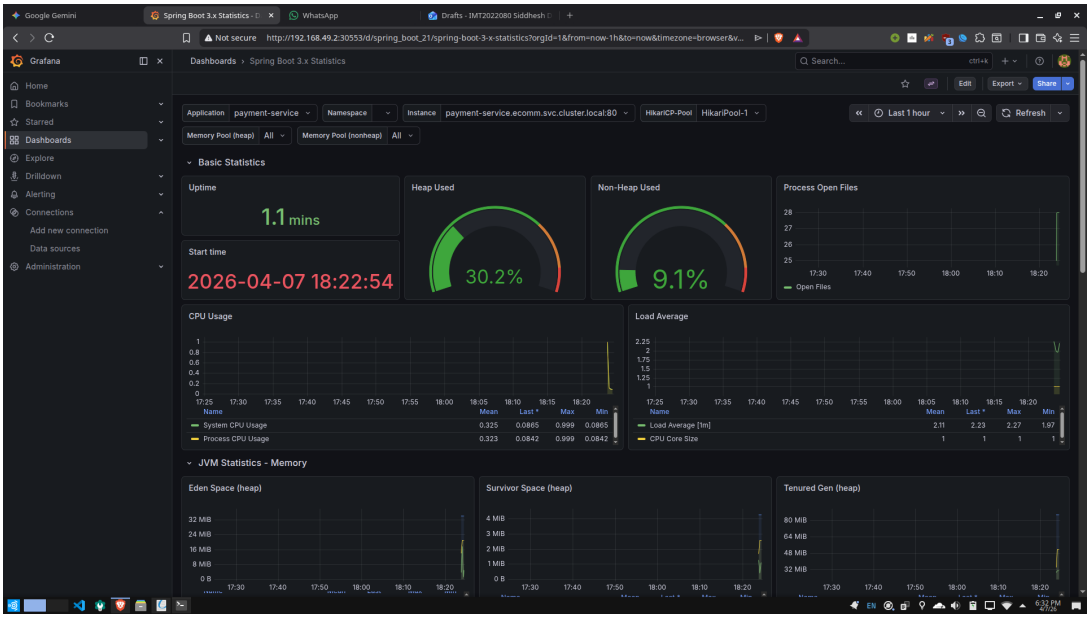

Figure 3: *Grafana Dashboard showing Spring Boot & JVM metrics*

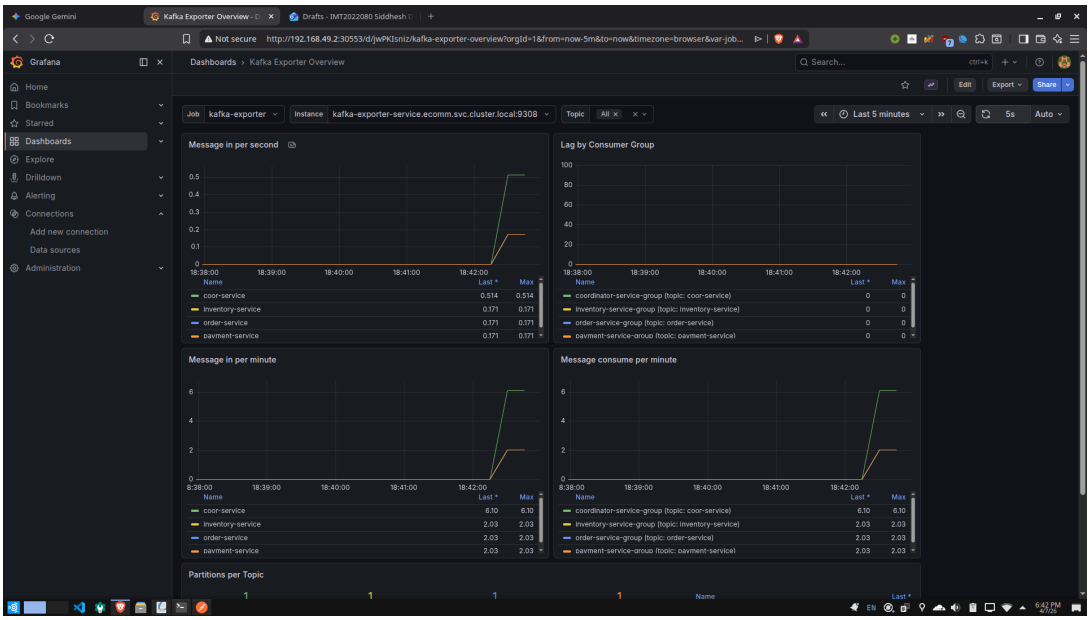

Figure 4: *Grafana Dashboard showing Kafka metrics*

6 Dynamic Autoscaling with KEDA & Prometheus

Moving beyond traditional CPU/Memory-based Horizontal Pod Autoscalers (HPA), this deployment incorporates intelligent, event-driven scaling using KEDA.

6.1 HTTP Traffic-Based Scaling (Coordinator)

The `coordinator` acts as the system's edge. It utilizes a KEDA `ScaledObject` that natively queries Prometheus. When the HTTP request rate exceeds 10 Requests Per Second (RPS), KEDA intercepts the metric scaling the deployment from 1 up to 5 replicas.

Listing 1: KEDA configuration for Prometheus-based HTTP scaling

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: coordinator-scaler
spec:
  scaleTargetRef:
    name: coordinator-deployment
  triggers:
    - type: prometheus
      metadata:
        serverAddress: http://prometheus-service.ecomm.svc.cluster.local:9090
        metricName: http_server_requests_seconds_count_rate
        query: sum(rate(http_server_requests_seconds_count{application="
            coordinator"}[1m]))
        threshold: "10"
```

6.2 Kafka Lag-Based Scaling (Worker Services)

The `order`, `inventory`, and `payment` services are decoupled background workers acting on Kafka events. KEDA is configured to scale these deployments identically via a Kafka trigger: if an application's specific Consumer Group lag exceeds 10 messages, KEDA proactively schedules new pod replicas to handle the burst.

Listing 2: KEDA configuration for Kafka Lag scaling

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: inventory-service-scaler
spec:
  scaleTargetRef:
    name: inventory-deployment
  triggers:
    - type: kafka
      metadata:
        bootstrapServers: kafka-service.ecomm.svc.cluster.local:9092
        consumerGroup: inventory-service-group
        topic: inventory-service
        lagThreshold: "10"
```

7 Conclusion and Future Enhancements

7.1 Conclusion

This project successfully demonstrates the fusion of robust core application paradigms (Distributed Transactions / 2PC) with modern, cloud-native deployment patterns. The integration

of OpenTelemetry/Jaeger and Prometheus creates a highly observable ecosystem where failures can be precisely traced. Furthermore, employing KEDA for event-driven scaling ensures the system dynamically reacts to both edge HTTP traffic and internal asynchronous Kafka backlogs, guaranteeing high throughput and efficiency.

7.2 Future Enhancements

Future iterations could explore:

- **Alerting:** Configuring Prometheus Alertmanager to notify teams via Slack or PagerDuty on scaling max-outs, critical Jaeger tracing failures, or sustained Kafka lag.
- **Cloud Migration:** Deploying this structured Minikube configuration onto a managed provider such as Amazon EKS, Azure AKS, or Google GKE.